

Computational Physics and Data Science Exercise 1: Simulating Lunar Orbits

Oscar Purrington

School of Physics, University of Bristol.

(Dated: October 21, 2025)

The Earth-Moon and Earth-Moon-Lunar Probe systems can be simulated using a numerical method for solving ODEs, namely the function `scipy.integrate.solve_ivp()`, using the Runge-Kutta 45 method. Using this we solve over a time span of 1 sidereal month to plot orbital trajectories of the moon around a stationary earth, and a lunar probe in a low lunar orbit around the moon. This method supplied accurate results for multiple initial conditions, allowing circular and elliptical orbits to be simulated. The runtimes of these simulations were considered and the amount of points integrated over modified.

I. INTRODUCTION

Simulating orbits is a common exercise in astrophysical computing and in numerical integration methods, providing experience in creating and solving differential equations using scientific coding modules. Writing robust code that can withstand different initial conditions for the orbits can be difficult and is effectively the object of this exercise. This exercise involves firstly determining the correct initial conditions to sustain various orbits, as well as all required physical constants, then writing code to set up and solve the various differential equations required, and finally visualising the orbits in a human understandable way. The code solves for various two-body orbits allowing insight into various orbital dynamics and behaviours under gravitational effects.

II. THEORY

Assuming a stationary earth, the following equation can be used to determine the orbit of the moon in earths gravitational field.

$$M_m \ddot{\mathbf{r}}_m = -\frac{M_e M_m G}{|\mathbf{r}_m|^2} \hat{\mathbf{r}}_m = -\frac{M_e M_m G}{|\mathbf{r}_m|^3} \mathbf{r}_m \quad (1)$$

where M_e is the mass of the earth, M_m the lunar mass, G the gravitational constant, where M_e is the mass of the earth, M_m the lunar mass, G the gravitational constant and $\mathbf{r}_m$ the vector of the lunar position from the center of the earth. Noting that $\mathbf{r}_m$ can be written in terms of x_m and y_m , we can write the derivatives of x_m , y_m , $v_{m,x}$, $v_{m,y}$ as

$$\frac{dv_{m,x}}{dt} = -\frac{M_e G x_m}{(x_m^2 + y_m^2)^{3/2}}; \quad \frac{dx_m}{dt} = v_{m,x}; \quad \frac{dv_{m,y}}{dt} = -\frac{M_e G y_m}{(x_m^2 + y_m^2)^{3/2}}; \quad \frac{dy_m}{dt} = v_{m,y}. \quad (2)$$

Finding the equivalent for the probe's orbit is trivial - we simply subtract the gravitational effect of the moon on the probe from the gravitational effect of the earth on the probe, allowing the equation of motion

$$M_p \ddot{\mathbf{r}}_p = -\frac{M_p M_e G \mathbf{r}_p}{|\mathbf{r}_p|^3} - \frac{M_p M_m G \mathbf{r}_{pm}}{|\mathbf{r}_{pm}|^3} \quad (3)$$

where $\mathbf{r}_p$ is the position vector of the probe relative to earth and $\mathbf{r}_{pm} = \mathbf{r}_p - \mathbf{r}_m$ is the position vector of the probe relative to earth, to be converted into the following components

$$\frac{dv_{p,x}}{dt} = -\frac{M_e G x_p}{(x_p^2 + y_p^2)^{3/2}} - \frac{M_m G x_{pm}}{(x_{pm}^2 + y_{pm}^2)^{3/2}}; \quad \frac{dx_p}{dt} = v_{p,x}; \quad \frac{dv_{p,y}}{dt} = -\frac{M_e G y_p}{(x_p^2 + y_p^2)^{3/2}} - \frac{M_m G y_{pm}}{(x_{pm}^2 + y_{pm}^2)^{3/2}}; \quad \frac{dy_p}{dt} = v_{p,y}. \quad (4)$$

To find the moon a probe orbital speeds, which are necessary for assigning initial conditons, we can equate centripetal force and the gravitational force to yield

$$v_m = \sqrt{\frac{GM_e}{r_{em}}}; \quad v_p = \sqrt{\frac{GM_m}{r_{mp}}} \quad (5)$$

where v_m , v_p are the moon and probe velocities, and r_{em} and r_{mp} are the earth-moon and moon-probe seperations respectively.

III. CODE EXPLANATION

The code begins with various imports

```
import time
import numpy as np
import scipy.integrate as si
import matplotlib.pyplot as plt
```

`time` is used to measure simulation runtimes, `numpy` is used for mathematical functions such as square roots, `scipy.integrate` is used to numerically solve the differential equations and `matplotlib.pyplot` is used to visualise the orbits. Then various constants were calculated and set. Many are known astronomical quantities such as `G` and the mass of the earth and moon `M_e`, `M_m`. Others, such as `V_m` and `V_p` are calculated using equation 5.

```
G = 6.6743e-11 # Gravitational Constant (m^3.kg^-1.s^-2)
M_e = 5.972e24 # Earth mass (kg)
M_m = 7.348e22 # Moon mass (kg)
R_em = 384400e3 # Average Earth-Moon Seperation (m)
R_mp = 1.737e6 + 100e3 # Probe-Moon seperation [Moon radius + LLO(100km)] (m)
V_m = np.sqrt(G * M_e / R_em) # Moon Orbital Speed (m.s^-1)
V_p = np.sqrt(G * M_m / R_mp) # Probe Orbital Speed (m.s^-1)
T_m = 2360591.597 # Moon orbital period [1 sidereal month] (s)
t_span = (0, T_m) # Time span for simulation [Multiple of sidereal months] (s)
t_coarse = np.linspace(
    t_span[0], t_span[1], 1001
) # Simulation points for a coarser integration [suitable for moon-only orbit] (n/a)
t_fine = np.linspace(
    t_span[0], t_span[1], 5001
) # Simulation points for moon-probe (n/a)
```

`t_span`, `t_coarse`, and `t_fine` are used as parameters in `si.integrate.solve_ivp()` - `t_span` being the time span over which the entire simulation occurs (one sidereal month), and `t_coarse` and `t_fine` being the number of points inside of this span at which the orbit is calculated. As 1001 points divide the orbit into time steps much smaller than the orbital period of the moon, `t_coarse` is perfectly enough to generate a smooth circular orbit without increasing computation time beyond necessary. When it comes to the probe, the orbital period is much closer to the time step. This causes various issues in smoothness, and leads to incorrect results. This is rectified by decreasing the step size until it is significantly smaller than the probe orbital period. 5000 time steps was a good round number that did not retain any jagged artifacts. This was used to define `t_fine`.

The function `moon_orbit_function(t, state)` takes in the initial values for the lunar orbit, namely its x and y positions and velocities and returns the derivatives from equation 2. This generates the acceleration components at each time step allowing the direction of the orbit to be determined. A similar function `probe_orbit_function(t, state)` works to determine the derivatives in equation 4, defining the orbit of the lunar probe in the moons frame.

The program also has two plotting functions. Firstly, a general function `plot()`, which produces a wide view of the orbits generated. The earth's position is always marked as a green circle.

```
plt.plot(0, 0, "o", color="green", markersize=8, label="Earth")
```

The program makes use of `matplotlib.pyplot` to draw the orbits, taking in the two solutions to either the moon orbit or the probe orbit. The user's selection in the main menu then determines if the probe orbit is considered or not.

```
axes.plot(moon_orbit_solution.y[0], moon_orbit_solution.y[1], label="Moon")

axes.legend(loc=1)
plt.show()

else:
    probe_x = moon_orbit_solution.y[0] + probe_orbit_solution.y[0]
```

```

probe_y = moon_orbit_solution.y[1] + probe_orbit_solution.y[1]

axes.plot(probe_x, probe_y, label="Probe")
axes.plot(moon_orbit_solution.y[0], moon_orbit_solution.y[1], label="Moon")

axes.legend(loc=1)
plt.show()

```

The second plotting function is `plot_zoom()`. This is a modification of the original `plot()` function, but it generates a second plot alongside the wide view when the user selects the probe orbit. This second plot is a much more zoomed in view of the moon-probe system, allowing the nuance in the orbit to actually be viewed, as the 100km low lunar orbit is much too small to be viewed when viewing the entire system. This is done by varying the x and y axis limits.

```

axes.set_xlim(0, 1e8)
axes.set_ylim(3.6e8, 4e8)

```

The `main()` function allows the user to input their choice of which part of the program to run, as well as calling all the required functions for the orbits, and the simulation timing function. A simple text-based main menu is displayed and the user can pick 1 to generate and display the moon-earth orbital system and 2 to do the same with the moon-earth-probe orbital system. Inputting Q will end the program. For either option, the relevant initial conditions are defined.

```

state_init_moon, state_init_probe = (R_em, 0, 0, V_m), (
    R_mp,
    0,
    0,
    V_p,
)

```

These states specify the moon and probe start at their maximum $+x$ displacement with an initial velocity in the $+y$ direction. The `si.solve_ivp()` functions are then called to generate the points of each bodies orbit by numerically integrating over the given time span.

```

moon_orbit_solution = si.solve_ivp(
    moon_orbit_function,
    t_span,
    state_init_moon,
    t_eval=t_fine,
    rtol=1e-6,
    atol=1e-6,
)
probe_orbit_solution = si.solve_ivp(
    probe_orbit_function,
    t_span,
    state_init_probe,
    t_eval=t_fine,
    rtol=1e-6,
    atol=1e-6,
)

```

Each simulation calls `start_time = time.time()` before running and then parses this value into a `print` containing a second `time.time()` call which is made after the simulation is complete. The simulation length is then displayed to the user to aid in assessing optimisation.

```

print(f"\nTime Elapsed {time.time()-start_time:.3f}s\n")

```

IV. RESULTS & DISCUSSION

This section will work to explain the results obtained, as well as how the script was modified to the point it is at submission. `si.solve_ivp()` has two parameters that allow us to change the tolerances of the integration. These are `rtol` and `atol`. When leaving the tolerances at the default values, the integration is not accurate enough, and the orbit becomes unstable. This can be seen clearly in figures 1 and 2. The true allowed error can be calculated using the following equation

$$\text{error} = \text{atol} + \text{rtol} \cdot |y| \quad (6)$$

where y is the given state variable. Inputting our default values for the tolerances and the earth-moon distance as y yields an error $\approx 3.84 \times 10^5$ m. This means that the orbital positions can be incorrect by hundreds of kilometres. Solving this is as simple as using more precise tolerances.

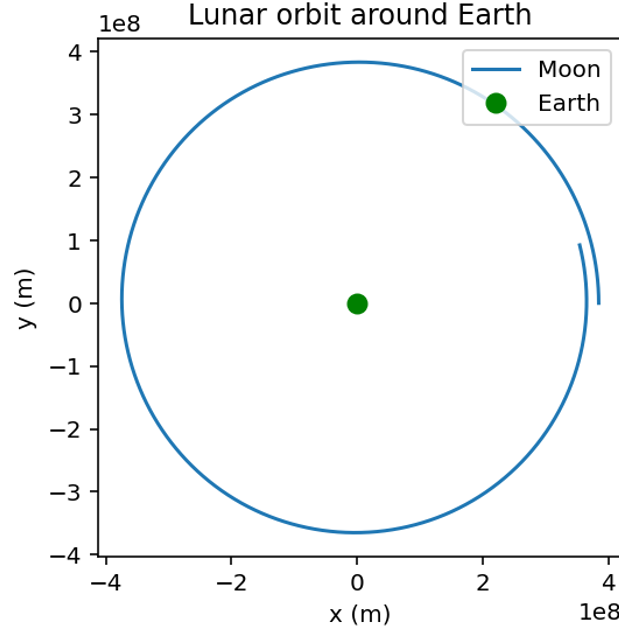


FIG. 1. The orbit generated over 1 sidereal month using default `si.solve_ivp()` `atol` and `rtol` values. Orbital deviation can be observed.

Values of `rtol` and `atol` that were found to produce accurate orbits in most common cases were `rtol`, `atol` = 1×10^{-6} . This can be seen by the consistent circle in figure 3. To test that these tolerances were good enough for the simulations at hand, generating elliptical orbits by modifying the initial velocities is a good method. The simulation produced accurate elliptical orbits over many orbital periods until it began to break down at 0.5 times the original initial velocity. Figure 4 shows a decently constant elliptical orbit with only some slight deviations at perihelion. Figure 5 shows a similar thing, but with more pronounced deviation at perihelion. The deviations only increase with a more eccentric orbit. As this program is designed to calculate circular orbits, the fact it can create such eccentric orbits with the tolerances given proves that the tolerance values are perfectly apt for the scope of the program.

The second part of the program generates a probes orbit around the moon. Initially, this was done using the same time steps as the moon program but this caused very jagged and incorrect orbits (figure 6). The time steps had to be decreased until they were significantly smaller than the orbital period of the probe. The probe was assumed to be orbiting at a low lunar orbit of 100km, as orbits smaller than this tend to be unstable. Assuming a circular orbit, the following was used to calculate the orbital period of the probe:

$$T = 2\pi \sqrt{\frac{r^3}{GM_m}} \quad (7)$$

where r is the orbital radius (i.e. lunar radius + orbital altitude) and M_m is the lunar mass. This gives an orbital period of ≈ 7064 s. 1000 time steps of a sidereal month is ≈ 2360 s, which is on the order of the orbital period,

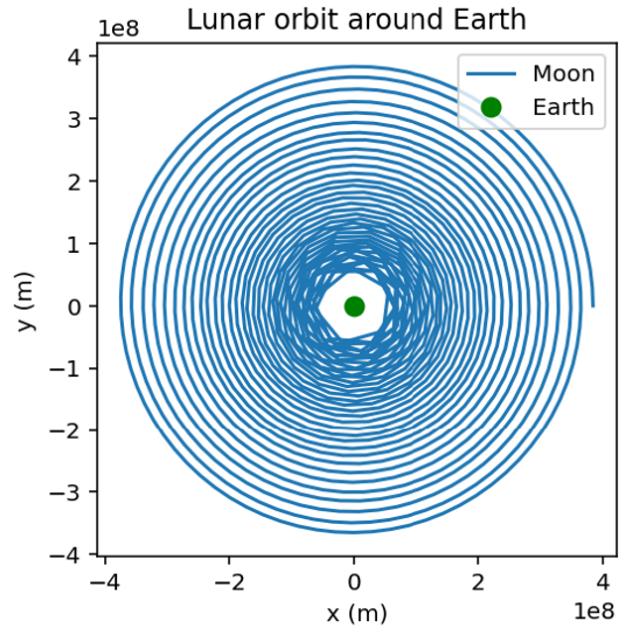


FIG. 2. The orbit generated over 12 sidereal months using default `si.solve_ivp()` `atol` and `rtol` values. Orbital deviation is very apparent.

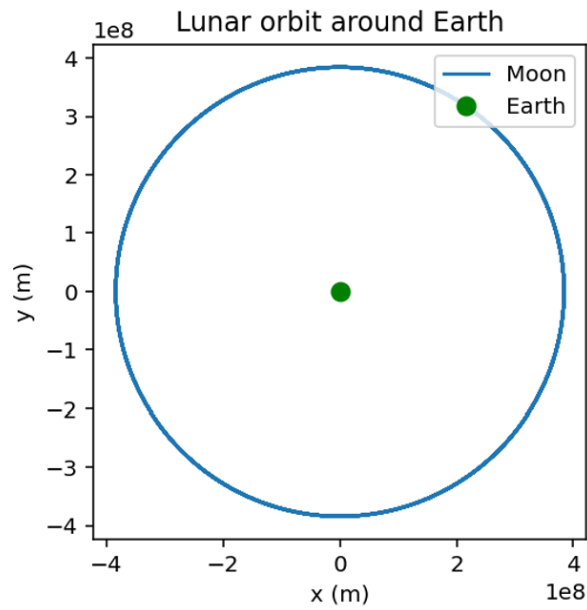


FIG. 3. The orbit generated over 12 sidereal months using default `si.solve_ivp()` `atol` and `rtol` values. Orbital deviation is very apparent.

therefore giving choppy and inaccurate results. Increase the number of time steps to 5000 reduced the time between positions to just ≈ 472 s, $15\times$ smaller than the orbital period. This was found to be accurate enough to generate a smooth orbit, shown in figure 7.

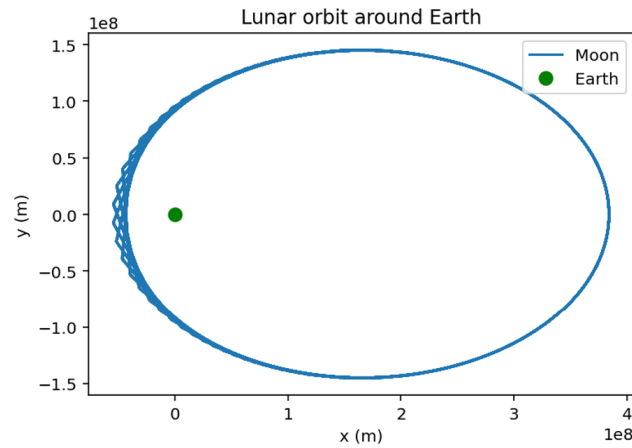


FIG. 4. An elliptical orbit generated using an initial velocity $0.5\times$ that of the original.

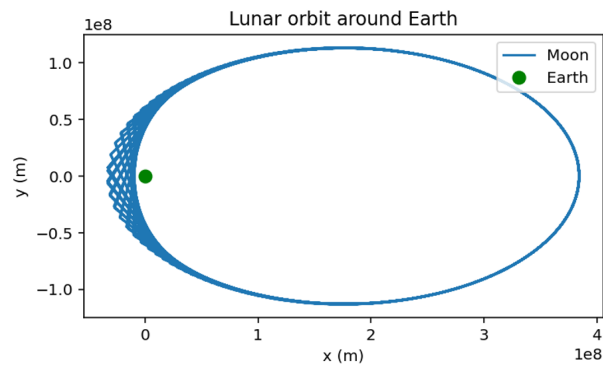


FIG. 5. An elliptical orbit generated using an initial velocity $0.4\times$ that of the original.

V. CONCLUSION

This simulation accurately and quickly produced earth-moon and earth-moon-probe orbital systems using the Runge-Kutta integration method. It emphasised accurate initial conditions, as well as an understanding of tolerances and code optimisation methods. It also provides proof that Newton's laws of gravity hold very well in these situations, as they were used to generate realistic initial conditions. These simulations do have some fallbacks, for example they fail to account for the earth's gravitational effects on the probe, which may cause some more complex orbital behaviour (turning this from two 2-body problems, to one 2-body and one 3-body problem - a much more complex relationship.) Any sort of orbital perturbations such as non-circular orbits and bodies was also entirely disregarded, as well as effects of the Sun's gravity. Future improvements would be modification of the code to render the second part of the problem as a true 3-body problem to allow for more accurate simulation, or perhaps improving the visualisation by producing an animation view of the orbits.

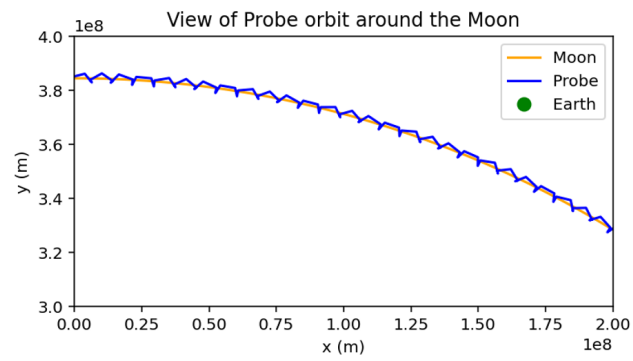


FIG. 6. A plot of the probe orbit made using 4000 time steps. This is $4\times$ bigger than the time steps used in the lunar orbit, but still retain many of the artifacts.

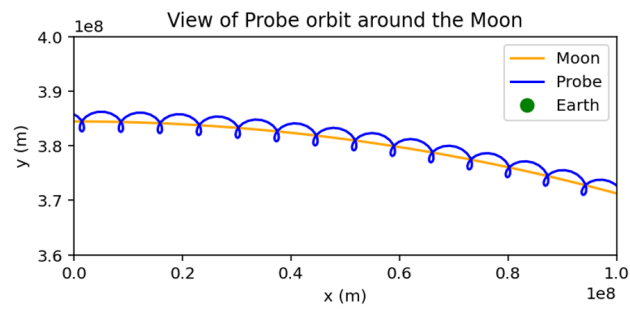


FIG. 7. A plot of the probe orbit made using 5000 time steps. This is enough to produce a nice defined orbit.